

Proyecto Final: EcoModelo - Calibración de una Sucesión Poblacional

Materia: Introducción a las Ciencias de la Computación

Profesor: Carlos Manuel Orrego Franco

Fecha: 19 de junio

Integrantes: Juan Sebastian Yopez - Juan Sebastian Pardo - Simon Ospina Gomez

1. Comprensión del Problema

El objetivo de este proyecto es calibrar un modelo matemático de crecimiento poblacional logístico discreto: $x(t+1) = x(t) + r \cdot x(t) \cdot (1 - x(t)/K)$. Dado un conjunto de datos observados [10, 12, 15, 19, 24, 30], nuestro algoritmo debe encontrar la mejor combinación de parámetros empíricos: la tasa de crecimiento (r) y la capacidad de carga (K), minimizando el error respecto a la realidad.

2. Modelamiento con Backtracking

Para evitar una búsqueda exhaustiva ineficiente (fuerza bruta), modelamos el problema como un árbol de decisiones usando la técnica de **backtracking con poda**.

- **Espacio de búsqueda (Candidatos):** Evaluar las combinaciones posibles en total para r [0.1, 0.2, 0.3, 0.4] y K [50, 80, 100, 120]. (4X4)

3. Bitácora de Desarrollo (Errores y Casos de Prueba)

Durante el desarrollo en Colab, enfrentamos y resolvimos los siguientes retos:

- **Cómo iniciar:** Estuvimos buscando como comenzar a desarrollar nuestro algoritmo en colab y decidimos utilizar un asistente de programación el cual fue gemini.
- **Inicio:** Estuvimos analizando el código para saber cómo funciona y todo lo que hay detrás realmente para lograr dar una correcta explicación de el mismo, añadimos comentarios e hicimos pruebas.
- **Bug inicial encontrado:** Olvidamos reiniciar la variable `error_acumulado = 0` al inicio del ciclo `for` de K . Esto causaba que los errores de iteraciones pasadas contaminaran la simulación actual. Se solucionó reiniciando el estado al inicio de cada nueva combinación.

- **Decisión de diseño:** Decidimos sumar el contador `pasos_con_poda` antes de evaluar los `if` de restricción, ya que cada intento de cálculo representa un costo de procesamiento real, independientemente de si resulta en poda o no.
- **Repartición:** Nos repartimos algunos trabajos como la bitácora, el informe y preguntas de defensa entre nosotros para agilizar el trabajo y dejamos las cosas del github y la presentación para hacer juntos ya que es lo de mayor importancia y complejidad.
- **Dudas con el profe:** Hablamos con el profe y le mostramos nuestras inquietudes, preguntamos cosas sobre el github y cosas generales para tener un buen proyecto.
- **Presentación:** realizamos la presentación, algo complejo por su código en overleaf pero no imposible y lo preparamos todo para culminar y subir nuestro proyecto a github.

5. Ejemplo Manual

Si probamos la combinación $r=0.4$ y $K=50$

- **Día 0:** Población = 10 (Dato real = 10). Error = 0.
- **Día 1:** Fórmula: $10 + (0.4 * 10 * (1 - 10/50)) = 13.2$. (Dato real = 12). Error del paso = 1.2. Error acumulado = 1.2.
- **Día 2:** Fórmula: $13.2 + (0.4 * 13.2 * (1 - 13.2/50)) = 17.08$. (Dato real = 15). Error del paso = 2.08. Error acumulado = 3.28.
- **Día 3:** Fórmula $17.08 + (0.4 * 17.08 * (1 - 17.08/50)) = 21.57$. (Dato real = 19). Error del paso = 2.57. **Error acumulado = 5.85.**
- **Resultado:** Como 5.85 es mayor que nuestro récord de 5.0 , **el algoritmo PODA en el día 3.** No calcula los días 4 y 5, ejecutando el retroceso.

6. Resultados y Conclusiones

- **Eficiencia:** El algoritmo demostró que el **backtracking** es significativamente superior a la fuerza bruta ciega. Al aplicar la poda, logramos un ahorro computacional medible, evitando calcular el árbol de posibilidades completo.
- **Solución encontrada:** El modelo logró calibrarse con éxito utilizando los parámetros $r = [0.3]$ y $K = [80]$, logrando el menor error total.
- **Conclusión general:** Entendimos que en Ciencias de la Computación, un buen diseño no solo se trata de llegar a la respuesta correcta, sino de saber descartar las respuestas incorrectas lo más rápido posible.